

Coexistence Without Coupling: Pattern C as a Standalone Architectural Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 7 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, Pattern C — the “separate concern” composition pattern within the CKS hybrid-systems framework — as a standalone architectural specification, so that downstream work can identify Pattern C deployments, distinguish them from Patterns A and B, and detect Pattern C anti-patterns without ambiguity.

Abstract

The hybrid-systems composition framework in the source paper supports three legitimate positions an adjacent AI component can occupy relative to a CKS substrate: as input to a cell (Pattern A), as a derived view of substrate content (Pattern B), or as a separate concern (Pattern C). The first two are coordinated flows — a cell consults the adjacent component during execution (Pattern A), or the substrate’s content is projected into the adjacent component as a derived view (Pattern B). Pattern C is operationally different from both: there is *no* coordination flow between the substrate and the adjacent component. The two systems coexist in the same deployment, may share infrastructure or organizational context, and yet are architecturally separate concerns from the point of view of substrate coordination. This note formalizes Pattern C as a standalone architectural specification through four operational components — no coordination coupling, substrate-cell boundary preserved through separation, no shared coordination state across the substrate’s authoritative categories, and composition requirements applicable when the separate concern includes AI — and through three sharpening properties (no-coordination-coupling, boundary-preserved, infrastructure-context-distinguishable) that distinguish Pattern C from its anti-patterns. The note also identifies the canonical anti-patterns into which “separate concern” deployments drift and supplies an operational test for whether a given hybrid deployment instantiates Pattern C in the source-paper sense.

1. Why Pattern C operationalization needs to be formalized as standalone

The hybrid-systems composition framework names three legitimate composition patterns, and prior derivation work in this series has formalized the broader framework (the three-pattern decomposition

itself) and the specialization of each pattern within it. Patterns A and B are coordinated flows. In Pattern A, a cell reads from the adjacent component during execution and writes results back to the substrate under its orchestration rule. In Pattern B, substrate content is indexed, embedded, or summarized into the adjacent component as a derived view that is regeneratable from substrate state. Both patterns specify what *flows* between substrate and adjacent component and what governs the flow.

Pattern C is operationally different. The defining property of Pattern C is that there *is no* coordination flow between the substrate and the adjacent component. The two systems are separate concerns from the point of view of substrate coordination, and the architecture’s responsibility is to specify what “separate” means precisely enough to be testable.

Without a standalone operationalization, “Pattern C” risks being read as a residual category — *anything that is not Pattern A or Pattern B* — which is exactly the reading that produces the anti-patterns discussed in §4 below. Two systems that share infrastructure (the same cloud account, the same network, the same datacenter) or organizational context (the same enterprise, the same user population, the same business unit) tend to acquire informal coordination flows over time — a shared lookup table, a script that scrapes one system’s state and pushes it into the other, a fine-tuned model whose training corpus includes substrate exports — that no orchestration rule governs. Pattern C as a standalone specification names precisely what such drifts violate: the no-coordination-coupling property that makes “separate concern” a defensible architectural position rather than a label applied to whatever happens to be deployed nearby.

The operationalization is consequential for enterprise deployments specifically, where CKS substrates routinely coexist with HR systems, payroll workflows, customer-service routing, observability pipelines, billing systems, and dozens of other coordinated artifacts that have their own coordination domains and need not — and architecturally must not — couple to the substrate’s coordination state. The hybrid-systems framework states the position; Pattern C operationalization states what holding the position requires.

2. The four operational components of Pattern C

A deployment’s relationship between a CKS substrate and an adjacent system is Pattern C if and only if four operational components hold simultaneously.

Component 1 — No coordination coupling between substrate and the separate concern. The substrate does not depend on the adjacent system for any of its authoritative coordination categories — decisions, rationale, conflicts, orchestration rules, or any of the categories the source paper names as substrate-as-source-of-truth content (§11.3). Symmetrically, the adjacent system does not depend on the substrate for its own coordination. There is no flow in either direction that the architecture must reason about: no consultation flow (which would make the relationship Pattern A), no derived-view flow (which would make it Pattern B), and no other flow that propagates coordination state between the systems.

Component 2 — Substrate-cell boundary preserved through separation. The substrate-cell boundary the architecture commits to applies inside the substrate’s coordination domain, and Pattern C

preserves it by leaving the adjacent system entirely outside that domain. The adjacent system does not write substrate content (it is not a writer at all, ungoverned or otherwise), does not bypass the cell layer (because there is no flow to bypass), and does not maintain ephemeral coordination state that should have lived in the substrate. The boundary is preserved not by the adjacent system being well-behaved with respect to it, but by the adjacent system having no operational relationship to it.

Component 3 — No shared coordination state across the substrate’s authoritative categories. The two systems may share storage, infrastructure, or organizational context (§3 below makes this distinction precise), but they do not share coordination state. If the adjacent system and the substrate both claim authority over the same decision, the same conflict, or the same rationale, the relationship is not Pattern C — it is either an unresolved Pattern B (one of them should be a derived view of the other) or an anti-pattern (both treated as authoritative for the same content, with no architectural account of how the two views are kept consistent). Pattern C requires the coordination domains themselves to be disjoint at the level of authoritative categories.

Component 4 — Composition requirements applicable when the separate concern includes AI. When the adjacent system contains AI components — a fine-tuned model serving non-coordination tasks, an LLM-driven customer-service workflow, an autonomous agent operating outside the substrate’s coordination scope — the composition requirements that govern AI-as-substrate-mediator do not lapse on the grounds that the AI is in a separate concern. Within the adjacent system’s own coordination domain (if it has one), AI must still operate as mediator, not as author or arbiter, of whatever substrate the separate concern itself maintains. “Separate concern” specifies that the AI is *outside the CKS substrate’s coordination domain*; it does not exempt the AI from operating as a mediator within whatever coordination domain it does serve. The architectural relationship is one of *separation between coordination domains*, not of exemption from architectural requirements within them.

3. What Pattern C operationalization forces beyond the broader composition framework

The broader hybrid-systems framework names three legitimate composition patterns and supplies the general shape of each; Pattern C operationalization forces four specifics that the general framework leaves implicit.

Explicit no-coupling specification. The general framework states that Pattern C systems “coexist without architectural coupling,” but the operationalization enumerates what coupling means by reference to the substrate’s authoritative categories: any flow that propagates substrate-authoritative state in either direction is coupling, regardless of how thin, how scheduled, or how technically incidental that flow may appear. The framework’s negative definition becomes a positive testable property.

Infrastructure and context sharing distinguished from coordination coupling. The most consequential clarification the operationalization introduces is the distinction between sharing infrastructure (the same cloud, the same network, the same database server, the same identity provider) or organizational context (the same enterprise, the same user population, the same business unit) and sharing

coordination state. Pattern C explicitly permits the former and explicitly forbids the latter. Two systems running on the same Kubernetes cluster are not coupled by the cluster; two systems exchanging coordination state through resources hosted on that cluster are. Two systems operating in the same enterprise are not coupled by the enterprise; two systems sharing decision provenance across the enterprise's substrates are. The operationalization gives deployment teams a vocabulary for what they are allowed to share without exiting Pattern C, and what they are not.

Substrate-cell boundary preserved by absence rather than by mediation. In Patterns A and B, the substrate-cell boundary is preserved by the cell layer mediating every flow — the cell consults the adjacent component (Pattern A), or the cell rebuilds the derived view (Pattern B). In Pattern C, the boundary is preserved by there being no flow to mediate. The operationalization makes this asymmetry explicit so that downstream work does not look for a Pattern C cell mediator that, by definition, has no work to do; the boundary discipline in Pattern C is one of separation, not of mediation.

AI-as-mediator role applies to separate-concern AI within its own substrate, not to the CKS substrate. The composition requirements that govern AI in CKS-mediated work apply to AI operating within any coordination domain — including the adjacent system's own coordination domain, if it has one. The operationalization clarifies that “separate concern” is a statement about *which* coordination domain the AI operates within, not a statement that the AI is outside any coordination domain at all. A separate concern containing an autonomous agent that writes to its own substrate without human governance is not exempted from the composition requirements simply by being a separate concern from the CKS substrate's standpoint; the requirements still apply within the separate concern's own architecture.

4. Anti-patterns specifically violating Pattern C

Pattern C deployments drift into anti-patterns through five canonical mechanisms, each of which violates one or more of §2's components.

Hidden bidirectional coupling — the canonical Pattern C violation. The substrate and the adjacent system update each other through automated processes — a script, a scheduled job, a database trigger, an event-stream subscription — that no human-authored orchestration rule governs. The systems were nominally separate concerns at the time the deployment was designed; the coupling accumulated as integrations were added “for convenience” and was never made architecturally visible. The fix is not to retroactively justify the coupling but to convert it into a legitimate Pattern A flow (the adjacent system's content is consulted by a cell under a rule) or Pattern B flow (one system's content becomes a derived view of the other's), at which point the architectural label is updated to reflect the flow that exists.

Ungoverned writer. The adjacent system writes substrate content directly — typically because the adjacent system shares storage with the substrate or has been granted write permissions in a shared identity context. The writes appear in substrate state without orchestration-rule trace, without provenance for the consultation that produced them, and without the authority constraints a cell would have applied. Pattern C explicitly forbids this: the adjacent system is a separate concern, which by definition means it does not write substrate state. If it does, it has either become a Pattern A consumer

(and must go through a cell under a rule) or it has bypassed the architecture entirely.

Pattern C with shared coordination state. Two systems labelled as separate concerns share coordination state through a “neutral” intermediary — a shared key-value store, a message queue, an event bus, a cache layer — without that intermediary going through legitimate Pattern A or Pattern B flows. The shared-state layer becomes an unnamed coordination surface that neither system governs and neither system can be inspected through. The fix is to recognize that the shared state is itself a substrate the architecture does not name and should not have, and either to bring it under one of the systems’ substrates as authoritative content or to remove the sharing.

Pattern C as substrate substitute. A deployment treats the adjacent system as authoritative for one or more of the substrate’s authoritative categories — decisions, rationale, conflicts — gradually displacing the substrate as the source of truth on those categories. This is the failure mode the source paper names in its discussion of substrate-as-source-of-truth (§11.3): an adjacent system that began as a separate concern accumulates content the substrate is also supposed to carry, and the deployment ends up with two stores claiming authority over the same coordination questions. Pattern C explicitly forbids this: separate concerns cannot share authoritative coordination categories with the substrate, by definition of the term.

Pattern-confusion violations. The most insidious Pattern C anti-patterns are the cases where an adjacent system has Pattern A or Pattern B flows but is *labelled* as Pattern C — typically because the original deployment was Pattern C, the flows accumulated later, and the label was not updated. A shared database from which cells routinely read at execution time is not Pattern C even if it began that way; it has become Pattern A. A search index that is rebuilt from substrate exports nightly is not Pattern C; it has become Pattern B. The fix is mechanical: the architectural label must reflect the flow that exists, not the deployment’s history. The label tracks operational reality; it does not certify intent.

The five anti-patterns share a common structural feature: an adjacent system that is described as Pattern C has, in operational fact, acquired one of the relationships Pattern C forbids — a flow, a writer, a shared state, a shared authoritative category, or a flow disguised by an out-of-date label. The operational test in §6 below is what makes the difference between description and operational fact testable.

5. What Pattern C is NOT

The operational specification is precise about what Pattern C *is*. It is equally important to state what Pattern C is not, because each of the following misreadings is a real misreading produced by the term’s surface meaning.

Not Pattern A. Pattern C is not “Pattern A with a quiet cell.” A Pattern A flow exists as soon as a cell consults the adjacent system under an orchestration rule, regardless of how often the consultation occurs. Pattern C requires the absence of consultation, not the rarity of it.

Not Pattern B. Pattern C is not “Pattern B with a small derived view.” A Pattern B flow exists as soon as substrate content is projected into the adjacent system as a regeneratable view, regardless of how thin

or how seldom-updated the projection is. Pattern C requires the absence of projection, not its smallness.

Not “any system that does not talk to CKS.” Pattern C is an architectural specification about the relationship between a CKS substrate and an adjacent system in a deployment context where coordination coupling could plausibly have arisen; it is not the negative space of “everything else in the world.” A system entirely unrelated to the substrate’s deployment is not in Pattern C; it is simply not part of the architectural picture. Pattern C names a specific positive property — coexistence without coordination coupling, in a context where coupling was a live possibility.

Not “shared infrastructure equals Pattern C.” The fact that two systems share a cloud account, a network, a datacenter, or a tenancy does not by itself place them in Pattern C. Infrastructure sharing is permitted within Pattern C but does not constitute Pattern C; what constitutes Pattern C is the absence of coordination flow, which infrastructure sharing neither implies nor prevents.

Not “exempts AI from mediator role.” Pattern C does not grant AI components in the adjacent system any exemption from operating as mediator within their own coordination domain. The architectural relationship between substrate and separate concern is one of domain separation, not of architectural permissiveness within the separate domain.

6. Operational test

A deployment instantiates Pattern C between a CKS substrate and an adjacent system if and only if all of the following are true at all times during the deployment’s existence.

1. **No-coordination-coupling.** No flow exists in either direction that propagates substrate-authoritative state — decisions, rationale, conflicts, orchestration rules, or any of the source paper’s authoritative substrate categories — between the substrate and the adjacent system. Coordination flows that do exist are Pattern A (cell-mediated consultation) or Pattern B (substrate-derived view); their existence excludes Pattern C for the affected scope.

2. **Boundary-preserved.** The substrate-cell boundary is preserved with respect to the adjacent system: the adjacent system does not write substrate content, does not bypass cells, and does not maintain coordination state that should have lived in the substrate. The boundary is preserved by absence of relationship, not by mediation.

3. **Infrastructure-context-distinguishable.** The deployment can answer the question “what do the two systems share?” with a list that includes infrastructure (cloud, network, storage, identity) and organizational context (enterprise, business unit, user population) and *does not include* any of the substrate’s authoritative coordination categories. The distinction is testable: the listing exists, and items that name authoritative coordination categories are absent from it.

A one-sentence test summarizes the three properties together: a deployment is in Pattern C if and only if the two systems coexist without any flow that propagates substrate-authoritative state, with the substrate-cell boundary preserved by separation, and with shared infrastructure and organizational context distinguishable from shared coordination state.

A deployment that fails any of (1)–(3) may be a useful deployment, and may even be governed in some other sense, but is not a Pattern C composition in the source-paper sense. The most common failure is (1) — a coordination flow that accumulated after the deployment was designed — and the architectural response is not to reclassify the deployment as “Pattern C with caveats” but to recognize that the flow has converted the relationship into Pattern A or Pattern B, and to update the deployment’s documented architecture accordingly.

7. Why naming Pattern C operationalization as standalone matters

Pattern C is the composition pattern that does the most architectural work for the least architectural visibility. Pattern A flows are visible in cell rules; Pattern B flows are visible in derived-view pipelines; Pattern C is visible only by what is *not* there. A deployment in Pattern C looks the same as a deployment whose Pattern A or Pattern B flows have not yet been built — until informal flows accumulate and the distinction matters retroactively.

Naming Pattern C as a standalone architectural specification, with the four operational components and the three sharpening properties, is what makes Pattern C inspectable. A team reviewing a deployment can now ask the operational test’s three questions explicitly and document the answers, rather than relying on the absence of evidence to imply the absence of coupling. The operationalization is what allows the architectural label to track operational reality, which is what allows the broader hybrid-systems framework to remain defensible across the lifetime of an enterprise deployment that gains and loses adjacent systems over time.

This note continues the fifth tier of the composition-pair phase of the derivation series, parallel to the prior fifth-tier notes that operationalize Pattern A (cell-to-adjacent consultation) and Pattern B (substrate-derived view) as standalone specifications. The next note in the tier closes the phase by establishing three-patterns coherence — the property that the three operationalizations, taken together, exhaust the legitimate composition space, with no fourth pattern available for adjacent systems that fit none of the three.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Coexistence Without Coupling: Pattern C as a Standalone Architectural Specification in the Coordination Knowledge Substrate Pattern*. 7 May 2026. ORCID: 0009-0004-8065-3235.